07 ARRAYS AND STRINGS • THEORY

02 array decay and pointer equivalence

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

02 — Array-to-Pointer Decay and Subscript Equivalence

1. Subscript Identity (`arr[i] == *(arr + i)`)

By definition in the C++ standard (`[expr.sub]`), `arr[i]` is identical to `*(arr + i)`.

2. Why Arrays Cannot Be Assigned (`arr1 = arr2` Fails)

An array name `arr` is **not** a modifiable pointer variable; it is a non-modifiable lvalue designating the array object itself. Therefore:

- `arr1 = arr2;` -> **Compilation Error** (invalid array assignment).
- `arr++;` -> **Compilation Error** (lvalue required as increment operand).

3. Passing Arrays to Functions

When passed to a function `void print_array(int arr[], int n)`, the parameter `int arr[]` is adjusted by the compiler to `int* arr`. Consequently, `sizeof(arr)` inside the function returns `sizeof(int*)` (8 bytes), **not** the byte size of the caller's array! Always pass the element count `int n` alongside a raw array, or use `std::span<const int>` (C++20/23) or `std::array<int, N>`.